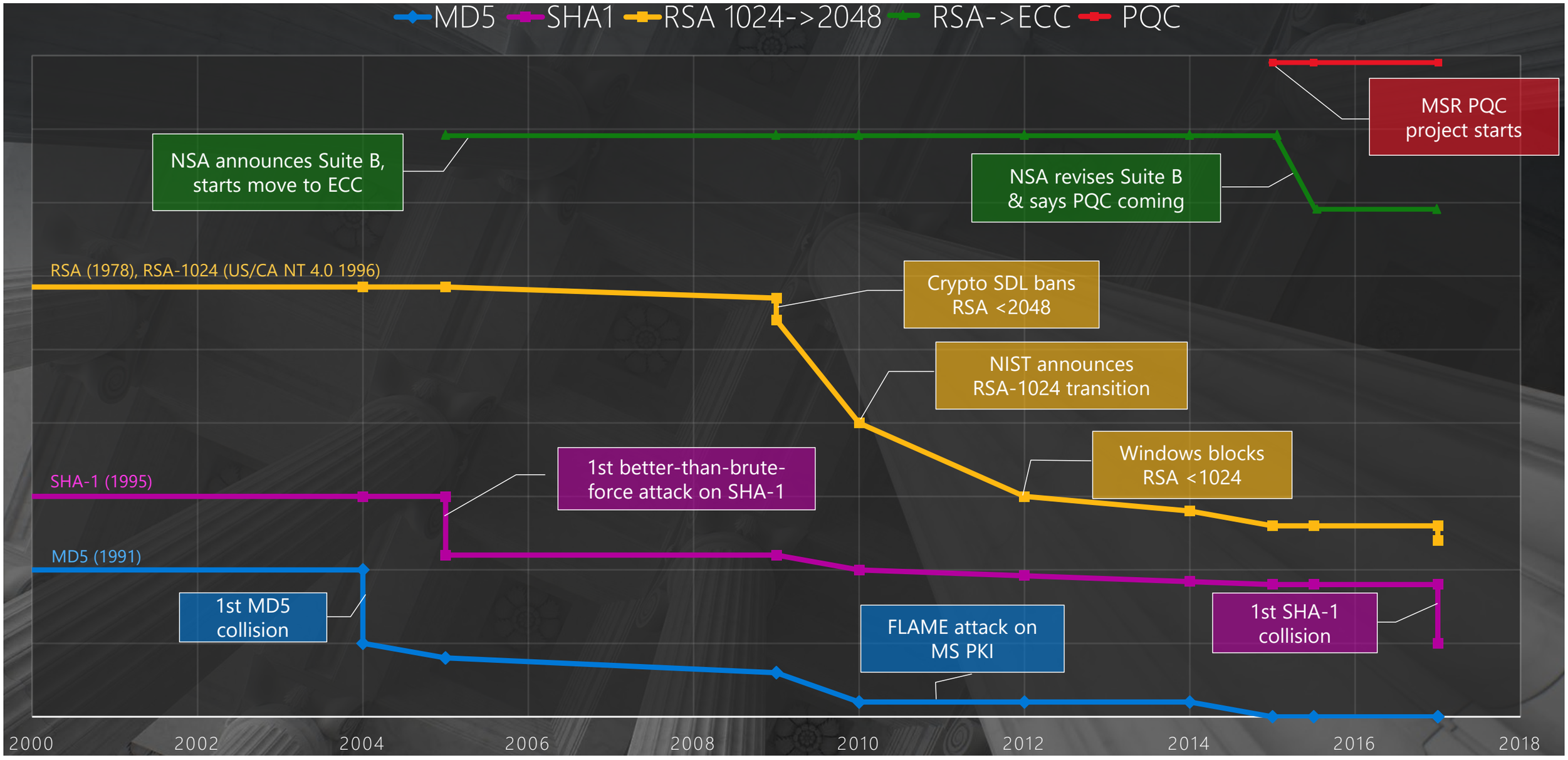
Hardware Security

硬體安全

Post Quantum Cryptography

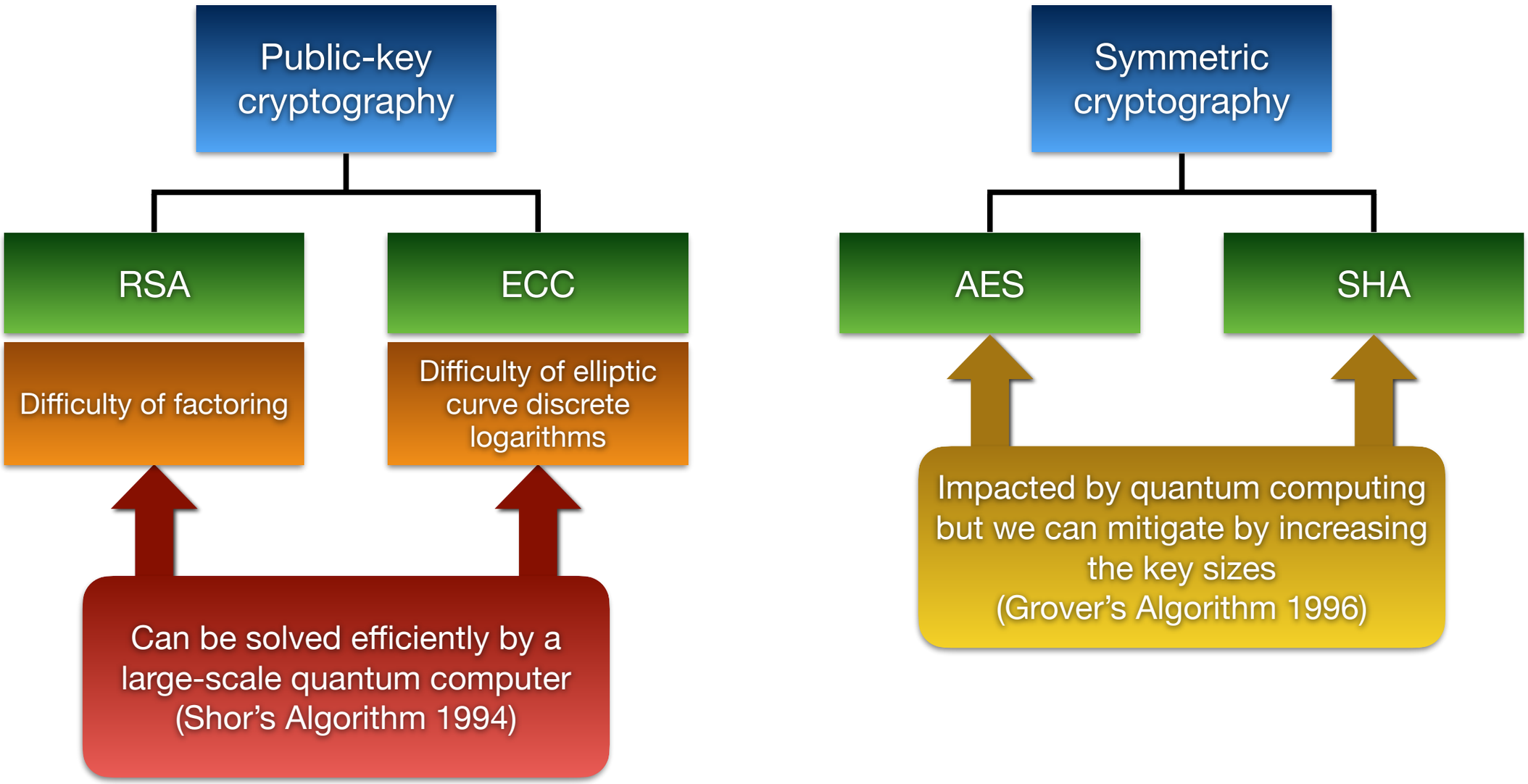
Wei-Chih Hong

Relative Strength of Crypto Algorithms over Time



Brian A. LaMacchia, The PQC Transition: Making Progress, But Still a Long Road Ahead, 2nd PQC Standardization & Migration Workshop

Quantum Threat to Today's PKCS



How soon will it arrive?

ECDLP in $E(\mathbb{F}_p)$ simulation results					Factoring of RSA modulus N interpolation from [21]		
$\lceil \log_2(p) \rceil$ bits	#Qubits	#Toffoli gates	Toffoli depth	Sim time sec	$\lceil \log_2(N) \rceil$ bits	#Qubits	#Toffoli gates
110	1014	$9.44 \cdot 10^9$	$8.66 \cdot 10^9$	273	512	1026	$6.41 \cdot 10^{10}$
160	1466	$2.97 \cdot 10^{10}$	$2.73 \cdot 10^{10}$	711	1024	2050	$5.81 \cdot 10^{11}$
192	1754	$5.30 \cdot 10^{10}$	$4.86 \cdot 10^{10}$	1 149	–	–	–
224	2042	$8.43 \cdot 10^{10}$	$7.73 \cdot 10^{10}$	1 881	2048	4098	$5.20 \cdot 10^{12}$
256	2330	$1.26 \cdot 10^{11}$	$1.16 \cdot 10^{11}$	3 848	3072	6146	$1.86 \cdot 10^{13}$
384	3484	$4.52 \cdot 10^{11}$	$4.15 \cdot 10^{11}$	17 003	7680	15362	$3.30 \cdot 10^{14}$
521	4719	$1.14 \cdot 10^{12}$	$1.05 \cdot 10^{12}$	42 888	15360	30722	$2.87 \cdot 10^{15}$

Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms, Roeteller et al., Asiacrypt 2017.

2023

Introduce parallelization of quantum computations.

2023 is all about pushing speed in quantum workflows by introducing parallelization in the Qiskit Primitives.

2024

Expand the utility of quantum computing.

We will improve the quality and speed of quantum circuits to allow running 5,000 gates with parametric circuits.

2025

Demonstrate quantum- centric supercomputing.

In 2025, we will enhance the quality of quantum circuits to allow running 7,500 gates and bring together modular processors, middleware, and quantum communication to demonstrate the first quantum-centric supercomputer.

2027

Scale quantum computing.

We will scale qubits, electronics, infrastructure, and software to reduce footprint, cost, and energy usage. The quality of quantum circuits will improve to allow running 10,000 gates.

2029

Deliver a fully error-corrected system.

We will bring users a quantum system with 200 qubits capable of running 100 milion gates.

2030+

Deliver quantum-centric supercomputers with 1,000's of logical qubits.

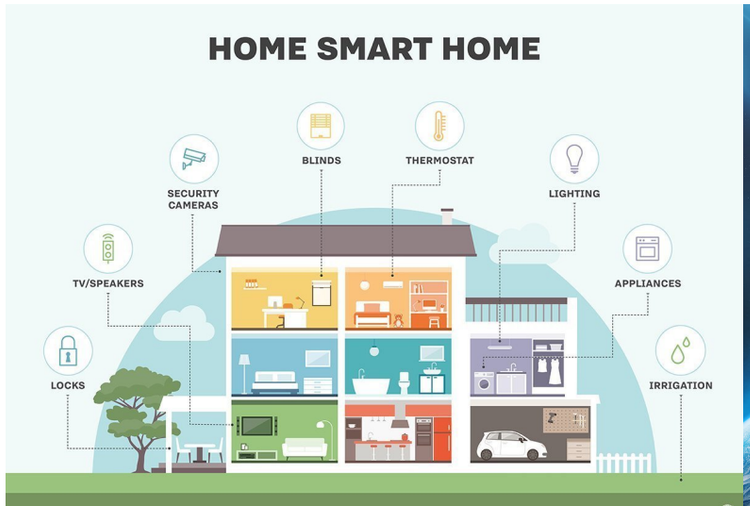
Beyond 2033, quantum-centric supercomputers will include thousands of qubits capable of running 1 billion gates, unlocking the full power of quantum computing.

https://www.ibm.com/roadmaps/quantum/

Future Quantum Computers are a Threat Today

- Even if a cryptographically-relevant quantum computer (CRQC) is a decade away…
- Record now, exploit later.
- Long-term security is an essential requirement for many contemporary applications and systems.
- Recovering the private keys of these systems could maliciously “update” the firmware in the devices.

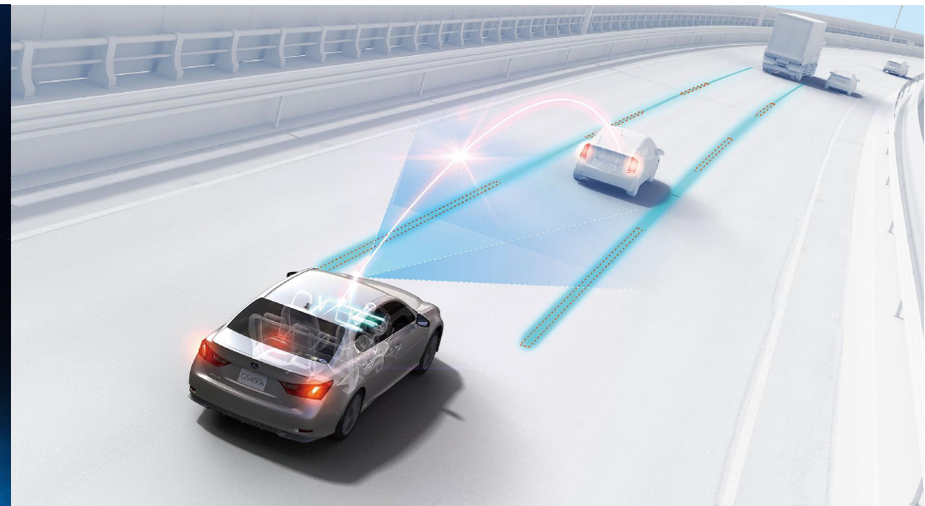
10~30 years



>15 years



5~25 years



Post Quantum Cryptography

- Based on problems which have no polynomial-time attacks (e.g. Grover's or Shor's algorithms) with quantum computers.
  - NP-hard problems (?)
- Should be able to run on today's devices.
- Efficiency in implementations
  - Software / hardware
- PQC ≠ quantum cryptography!

Categories of PQC

- **Hash-based signatures (Lamport, 1979)**
  - SPHINCS+, XMSS
- **Code-based encryptions (McEliece, 1978)**
  - McEliece
- **Multivariate cryptography (松本 & 今井, 1988)**
  - UOV, Rainbow
- **Lattice-based cryptography (Ajtai, 1996)**
  - Kyber, Dilithium, Falcon
- **Isogeny-based cryptography (De Feo, Jao, and Plut, 2011)**
  - SIDH, SIKE

NIST PQC Standardization

2016	2017	2019	2020	2022	2024
NIST initialized the PQC standardization process by announcing the call for proposals.	**69** of the 82 submissions accepted as 1st round candidates.	**29** algorithms selected as 2nd round candidates.	NIST announced **7** finalists for the 3rd round.	Identified **4** algorithms for standardization. Announced 4th round candidates and call for additional signature schemes.	Published the **FIPS-203, 204, 205** standards for PQC.

https://csrc.nist.gov/projects/post-quantum-cryptography

NIST PQC Round 4

- 4 additional encryption algorithms to continue to Round 4
  - BIKE
  - HQC
  - Classic McEliece
  - SIKE
- No signature algorithms were selected for Round 4, but…

NIST Call for Additional Signature Schemes

- NIST received 40 “complete and proper” submissions before June 1, 2023.

Code-based	Lattice-based	MPCitH	Multivariate	Symmetric-based	Isogeny-based
Enhanced pqsigRM	EagleSign	CROSS	3WISE	AIMer	SQIsign
Fuleeca	EHTv3 and EHTv4	MIRA	Biscuit	Ascon-Sign	
LESS	HAETAE	MiRitH	DME-Sign	FAEST	
MEDS	Hufu	MQOM	HPPC	SPHINCS-alpha	
Wave	Raccoon	PERK	MAYO		
	SQUIRRELS	RYDE	PROV		
	HAWK	SDitH	QR-UOV		
			SNOVA		
			TUOV		
			UOV		
			VOX		

Other

ALTEQ

eMLE-Sig 2.0

KAZ-SIGN

Preon

Xifrat1-sign.I

FIPS PQC Standards

- **Published on August 13, 2024**

FIPS-203	FIPS-204	FIPS-205
ML-KEM	ML-DSA	SLH-DSA
CRYSTALS-Kyber	CRYSTALS-Dilithium	SPHINCS+
Lattice	Lattice	Hash
Key establishment	Digital signature	Digital signature
Security categories 1, 3, 5	Security categories 2, 3, 5	Security categories 1, 3, 5

Hash-based Cryptography

- Cryptographic hash functions (CHF) that map arbitrary length messages to fixed length digests
- Only signature schemes available, no encryption
- **Minimum security assumption: secure CHF**
  - Second pre-image resistant

Lamport-Diffie One-Time Signatures

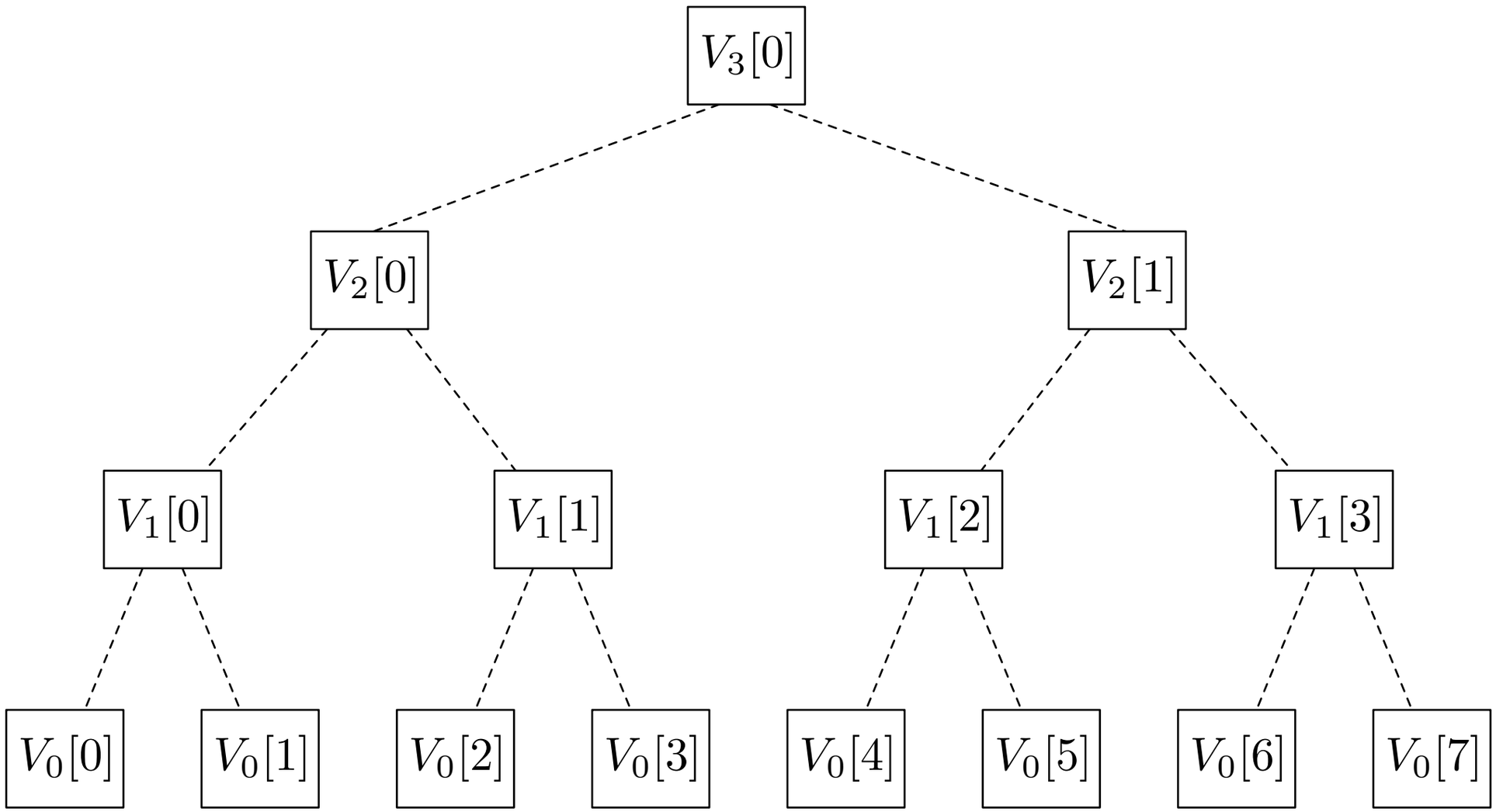
- Given a security parameter n, the set of n-bit vectors $U_n = \{0,1\}^n$ and a one-way function $h : U_n \to U_n$.
- **Secret key**: Generate $2n \times n$-bit vectors $X = (x_{(0,0)}, x_{(0,1)}, x_{(1,0)}, x_{(1,1)}, \ldots, x_{(n-1,0)}, x_{(n-1,1)})$
- **Public key**: Compute $Y = (y_{(0,0)}, \ldots, y_{(n-1,1)})$, $\forall y_{(i,j)} = h(x_{(i,j)})$
- Signing a message $M = (m_0, m_1, \ldots, m_{n-1})$ is to generate the signature $\sigma = (x_{(0,m_0)}, \ldots, x_{(n-1,m_{n-1})})$
- How about the verification of a signature?

Inefficiencies of the LD-OTS Scheme

- Each key pair can only be used once.
- Both the private and public key sizes are huge.
- Signature size is also huge.

Merkle Tree (Hash Tree)

- Efficient and secure verification of the contents of a large data structure.



Merkle Signature Scheme

- Let $h : \{0,1\}^* \longmapsto \{0,1\}^n$ be a hash function with security parameter n.
- Fix height H and generate 2^H LD-OTS key pairs (X_i, Y_i) with $0 \leq i < 2^H$.
- **Notation**: $V_i[j]$ with $0 \leq i \leq H$ and $0 \leq j < 2^{H-j}$
- **Leaf nodes**: $V_0[j] = h(Y_j)$
- **Inner nodes**: $v_i[j] = h(V_{i-1}[2j] \parallel V_{i-1}[2j+1])$ for $0 < i \leq H$ and $0 \leq j < 2^{H-j}$
- **Public key**: the root $V_H[0]$

Other Techniques for Hash-based Crypto

- **Winternitz OTS (WOTS)**
  - Signing a ℓ-bit message m at a time by publishing $h^m(x)$
  - $(x, h^\omega(x))$ is the private and public key pair, where $\omega = 2^\ell$
- **SPHINCS$^+$**
  - A hypertree with the leaf layer being FORS keys for message signing
  - Enabling "stateless" signature scheme by selecting the leaf with pseudo random function (PRF)

SLH-DSA Parameter Sets

	n	h	d	h'	a	k	lg_w	m	security category	pk bytes	sig bytes
SLH-DSA-SHA2-128s SLH-DSA-SHAKE-128s	16	63	7	9	12	14	4	30	1	32	7 856
SLH-DSA-SHA2-128f SLH-DSA-SHAKE-128f	16	66	22	3	6	33	4	34	1	32	17 088
SLH-DSA-SHA2-192s SLH-DSA-SHAKE-192s	24	63	7	9	14	17	4	39	3	48	16 224
SLH-DSA-SHA2-192f SLH-DSA-SHAKE-192f	24	66	22	3	8	33	4	42	3	48	35 664
SLH-DSA-SHA2-256s SLH-DSA-SHAKE-256s	32	64	8	8	14	22	4	47	5	64	29 792
SLH-DSA-SHA2-256f SLH-DSA-SHAKE-256f	32	68	17	4	9	35	4	49	5	64	49 856

- **For Comparison, the sizes of the pk and sig for 256-bit ECDSA are 32 and 64 bytes respectively.**

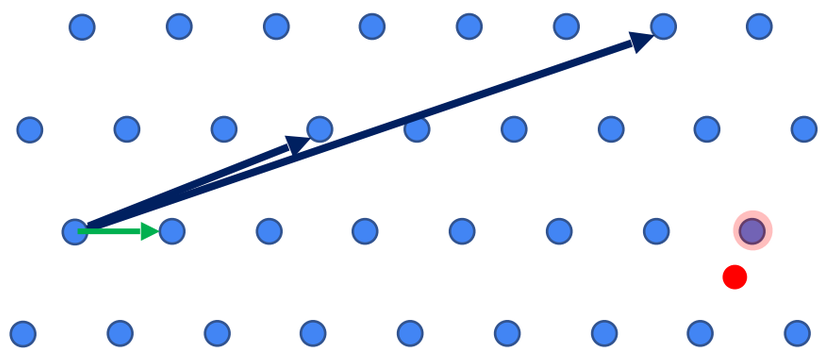
Lattice-based Cryptography

- Encryption and signature systems are both feasible and secure.
- Most operations are efficient and can be parallelized.
- Reasonably large public and private keys
- Reasonably large ciphertexts or signatures

What Is a Lattice?

- Given a basis $\mathbf{B} = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n\}$, the space of all possible integer combinations of the vectors in $\mathbf{B}$ is called a lattice.

- Lattice problems are hard problems that in such spaces, e.g. SVP and CVP.



Shortest Vector Problem (SVP)
To find the shortest vector in a given basis $\mathbf{B}$

Closest Vector Problem (CVP)
To find the closest vector to a target $\mathbf{t}$ in a given basis $\mathbf{B}$

Learning with Error (LWE)

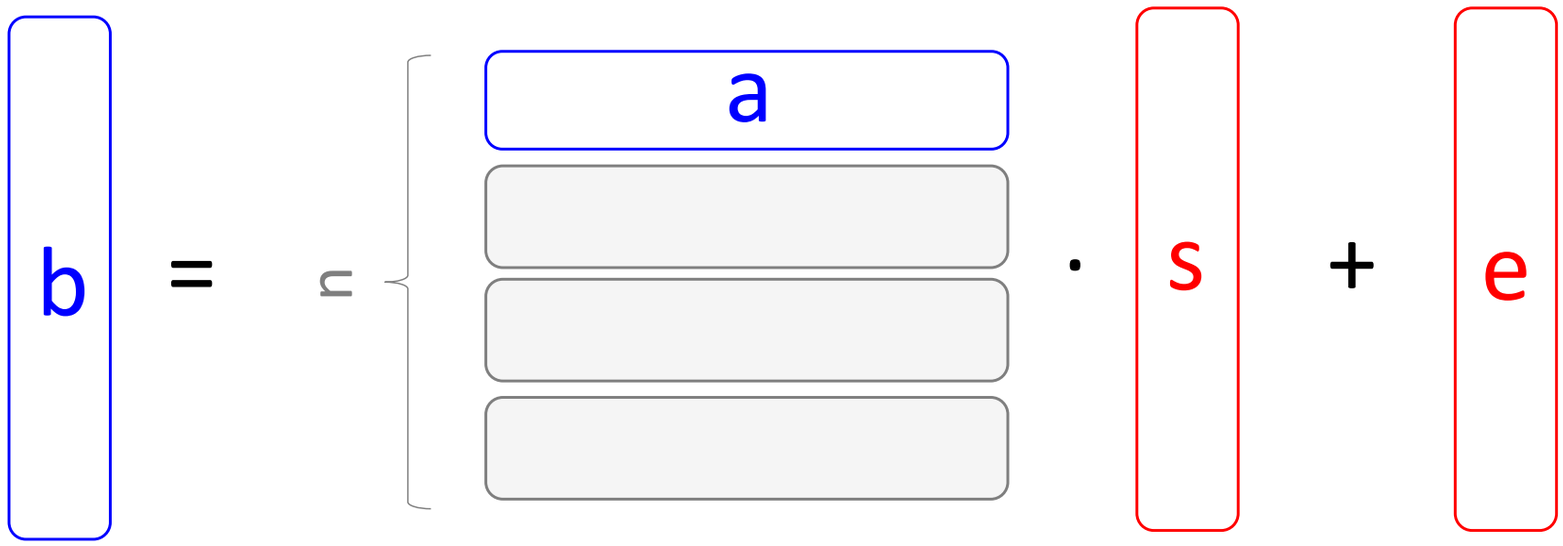
- Given some uniform random $a, b = a \cdot s + e$:
  - **Search LWE**: decode the noisy product b, i.e. recover s from a, b for "small" e
  - **Decision LWE**: distinguish b from uniform random
- Plain LWE: $a \leftarrow \mathbb{Z}_q^n, s \leftarrow U$ or $\chi_\sigma^n, e \leftarrow \chi_\sigma, b \in \mathbb{Z}_q$

$$\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \vdots \\ \mathbf{a}_m \end{bmatrix} \cdot \mathbf{s} + \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{bmatrix} \qquad \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$$

Practical Ring-LWE

- **Size of the pk (matrix $\mathbf{A}$) would be larger than 1MB in order to make plain LWE secure.**

- Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be the polynomial ring.
- Given some uniform random $a \in R_q$ and $s, e \leftarrow \chi_\sigma^n$: recover s from $b = a \cdot s + e$



Polynomial Multiplication in R_q

- $\boldsymbol{k} = (2, 1, 4, 0)$
- $\boldsymbol{s} = (1, 3, 4, 1)$

- **Task:** $\boldsymbol{z} = \boldsymbol{s} * \boldsymbol{k} = (3, 0, 2, 0)$

multiply-then-reduce

+			0	0	0	0	0
+		4	12	16	4		4
+	1	3	4	1			1
+	2	6	8	2			2
	2	7	15	18	17	4	0

mod 5/mod $x^{(n+1)}$

z	3	0	2	0

in-place multiplication

+	0	0	0	0	0
+	12	16	4	-4	4
+	4	1	-1	-3	1
+	2	-2	-6	-8	2
	18	15	-3	-15	

mod 5

z	3	0	2	0

Benefits of R-LWE

- **Compact representation of matrix A**
  - For LWE, the matrix $\mathbf{A} \in \mathbb{Z}_{4093}^{640\times 256}$.
  - For R-LWE, the polynomial $\mathbf{a} \in \mathbb{Z}_{4093}[X]/(X^{256}+1)$.

- **Polynomial multiplication can be accelerated using number theoretic transform (NTT).**
  - $\mathcal{O}(n^2) \longrightarrow \mathcal{O}(n \log n)$

Number Theoretic Transform

- Evaluate $\mathbf{a}(x)$ at ω^i, where ω is the n-th primitive root of unity.

$$\tilde{\mathbf{a}}[i] = \sum_{j=0}^{n-1} \mathbf{a}[j]\omega^{ij} \bmod q, \;\; i = 0, 1, \ldots, n-1$$

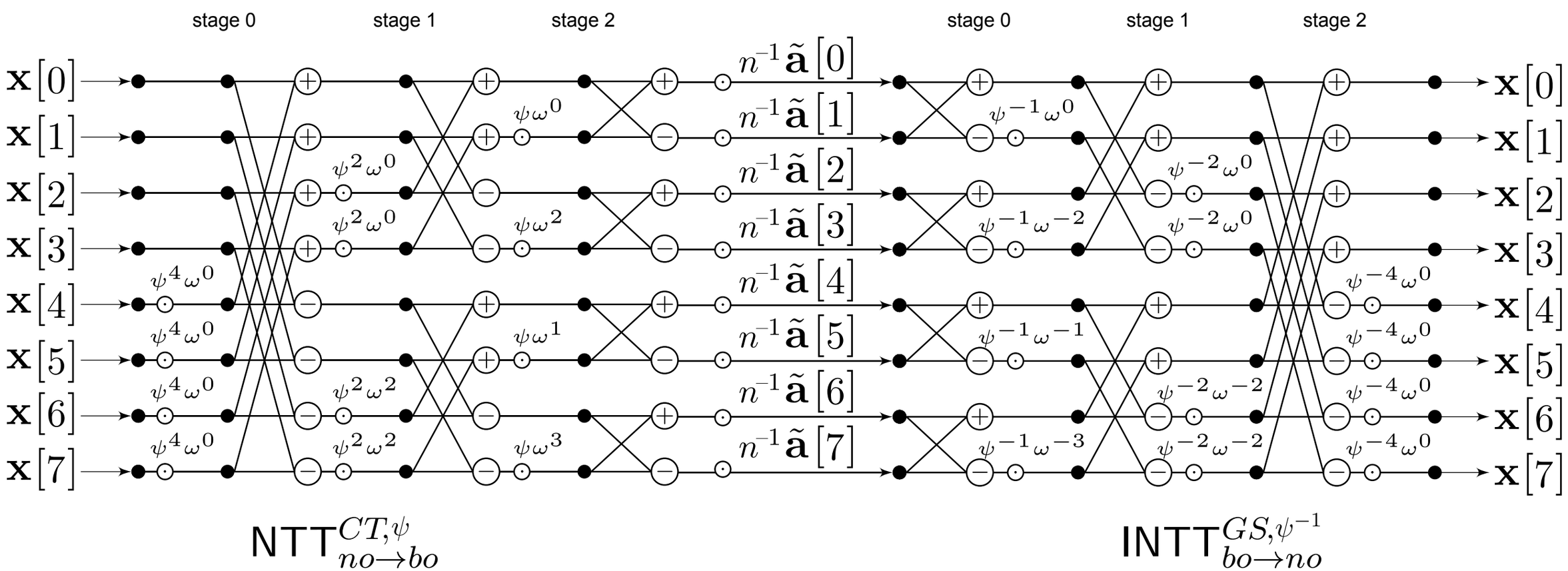


Fig. 1. Signal flow graph for multiplication of a polynomial $\mathbf{x}$ by a pre-transformed polynomial $\tilde{\mathbf{a}} = \mathrm{NTT}^{CT,\psi}_{no\to bo}(\mathbf{a})$, using the $\mathrm{NTT}^{CT,\psi}_{no\to bo}$ and $\mathrm{INTT}^{GS,\psi^{-1}}_{bo\to no}$ algorithms.

Modular Arithmetics

- Keep the results in $[0,\ q\text{ -}1]$ by applying $\mathrm{mod}\ q$
- Additions and subtractions are easy, but the results of multiplications will require division if the mod operations are invoked directly.

Montgomery Multiplication

- **Montgomery form** $\bar{a} \equiv aR \mod N$, where $R > N$ and $\gcd(R, N) = 1$.
- Let $RR^{-1} \equiv 1 \mod N$.
  $\Rightarrow RR^{-1} = kN + 1$ for some integer k.
- When computing $c \equiv ab \mod N$, we compute $\bar{c} \equiv abR \equiv (aR)(bR)R^{-1} \equiv \bar{a}\bar{b}R^{-1} \mod N$ instead.
- We need an efficient way to compute $TR^{-1} \mod N$.

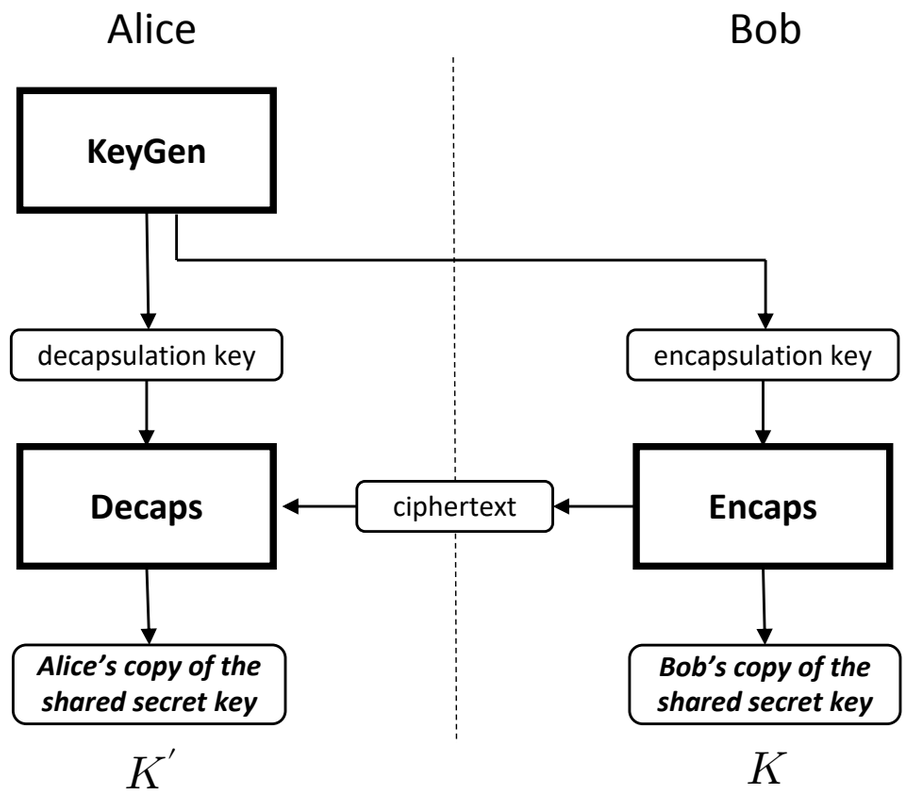
Montgomery Reduction

Data: $0 \leq T < RN$
Result: $t \equiv TR^{-1} \mod N$
$m \leftarrow (T \mod R) \cdot k \mod R$;
$t \leftarrow (T + mN)/R$;
if $t \geq N$ **then**
  $t \leftarrow t - N$;
end

- If we choose $R = 2^n$, then the division by R can be done with shifting.
- The reduction procedure requires only 2 multiplications.

FIPS 203 ML-KEM (CRYSTALS-Kyber)

- **Module-Lattice-based Key-Encapsulation Mechanism**
  - A KEM is a cryptographic scheme that, under certain conditions, can be used to establish a shared secret key between two communicating parties. This shared secret key can then be used for symmetric-key cryptography.
- Three functions
  - KeyGen( )
  - Encaps( )
  - Decaps( )



Construction of ML-KEM

- **K-PKE**
  - A deterministic public key encryption with 3 security level parameter settings.
  - Only a component of ML-KEM and shall not be used as a stand-alone crypto scheme.
- **ML-KEM**
  - Defines an IND-CCA2 secure probabilistic KEM scheme based on K-PKE

Parameter set	Decapsulation failure rate
ML-KEM-512	$2^{-138.8}$
ML-KEM-768	$2^{-164.8}$
ML-KEM-1024	$2^{-174.8}$

K-PKE.KeyGen( )

Algorithm 13 K-PKE.KeyGen(d)

Uses randomness to generate an encryption key and a corresponding decryption key.

Input: randomness $d \in \mathbb{B}^{32}$.
Output: encryption key $\text{ek}_{\text{PKE}} \in \mathbb{B}^{384k+32}$.
Output: decryption key $\text{dk}_{\text{PKE}} \in \mathbb{B}^{384k}$.

$(\rho, \sigma) \leftarrow \text{G}(d\|k)$ ▷ expand 32+1 bytes to two pseudorandom 32-byte seeds[1]
$N \leftarrow 0$
for ($i \leftarrow 0$; $i < k$; i++) ▷ generate matrix $\hat{\mathbf{A}} \in (\mathbb{Z}_q^{256})^{k \times k}$
    for ($j \leftarrow 0$; $j < k$; j++)
        $\hat{\mathbf{A}}[i,j] \leftarrow \text{SampleNTT}(\rho\|j\|i)$ ▷ j and i are bytes 33 and 34 of the input
    end for
end for
for ($i \leftarrow 0$; $i < k$; i++) ▷ generate $\mathbf{s} \in (\mathbb{Z}_q^{256})^k$
    $\mathbf{s}[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N))$ ▷ $\mathbf{s}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
    $N \leftarrow N + 1$
end for
for ($i \leftarrow 0$; $i < k$; i++) ▷ generate $\mathbf{e} \in (\mathbb{Z}_q^{256})^k$
    $\mathbf{e}[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N))$ ▷ $\mathbf{e}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
    $N \leftarrow N + 1$
end for
$\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s})$ ▷ run NTT k times (once for each coordinate of $\mathbf{s}$)
$\hat{\mathbf{e}} \leftarrow \text{NTT}(\mathbf{e})$ ▷ run NTT k times
$\hat{\mathbf{t}} \leftarrow \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}$ ▷ noisy linear system in NTT domain
$\text{ek}_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{\mathbf{t}})\|\rho$ ▷ run ByteEncode_{12} k times, then append $\hat{\mathbf{A}}$-seed
$\text{dk}_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{\mathbf{s}})$ ▷ run ByteEncode_{12} k times
return $(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}})$

K-PKE.Encrypt( )

Algorithm 14 K-PKE.Encrypt$(\text{ek}_{\text{PKE}}, m, r)$

Uses the encryption key to encrypt a plaintext message using the randomness r.

Input: encryption key $\text{ek}_{\text{PKE}} \in \mathbb{B}^{384k+32}$.
Input: message $m \in \mathbb{B}^{32}$.
Input: randomness $r \in \mathbb{B}^{32}$.
Output: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.

$N \leftarrow 0$
$\hat{\mathbf{t}} \leftarrow \text{ByteDecode}_{12}(\text{ek}_{\text{PKE}}[0 : 384k])$ ▷ run ByteDecode$_{12}$ k times to decode $\hat{\mathbf{t}} \in (\mathbb{Z}_q^{256})^k$
$\rho \leftarrow \text{ek}_{\text{PKE}}[384k : 384k + 32]$ ▷ extract 32-byte seed from ek_{PKE}
for ($i \leftarrow 0$; $i < k$; i++) ▷ re-generate matrix $\hat{\mathbf{A}} \in (\mathbb{Z}_q^{256})^{k \times k}$ sampled in Alg. 13
    for ($j \leftarrow 0$; $j < k$; j++)
        $\hat{\mathbf{A}}[i, j] \leftarrow \text{SampleNTT}(\rho \| j \| i)$ ▷ j and i are bytes 33 and 34 of the input
    end for
end for
for ($i \leftarrow 0$; $i < k$; i++) ▷ generate $\mathbf{y} \in (\mathbb{Z}_q^{256})^k$
    $\mathbf{y}[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(r, N))$ ▷ $\mathbf{y}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
    $N \leftarrow N + 1$
end for
for ($i \leftarrow 0$; $i < k$; i++) ▷ generate $\mathbf{e_1} \in (\mathbb{Z}_q^{256})^k$
    $\mathbf{e_1}[i] \leftarrow \text{SamplePolyCBD}_{\eta_2}(\text{PRF}_{\eta_2}(r, N))$ ▷ $\mathbf{e_1}[i] \in \mathbb{Z}_q^{256}$ sampled from CBD
    $N \leftarrow N + 1$
end for
$e_2 \leftarrow \text{SamplePolyCBD}_{\eta_2}(\text{PRF}_{\eta_2}(r, N))$ ▷ sample $e_2 \in \mathbb{Z}_q^{256}$ from CBD
$\hat{\mathbf{y}} \leftarrow \text{NTT}(\mathbf{y})$ ▷ run NTT k times
$\mathbf{u} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}}^\top \circ \hat{\mathbf{y}}) + \mathbf{e_1}$ ▷ run NTT^{-1} k times
$\mu \leftarrow \text{Decompress}_1(\text{ByteDecode}_1(m))$
$v \leftarrow \text{NTT}^{-1}(\hat{\mathbf{t}}^\top \circ \hat{\mathbf{y}}) + e_2 + \mu$ ▷ encode plaintext m into polynomial v
$c_1 \leftarrow \text{ByteEncode}_{d_u}(\text{Compress}_{d_u}(\mathbf{u}))$ ▷ run ByteEncode$_{d_u}$ and Compress$_{d_u}$ k times
$c_2 \leftarrow \text{ByteEncode}_{d_v}(\text{Compress}_{d_v}(v))$
return $c \leftarrow (c_1 \| c_2)$

K-PKE.Decrypt( )

Algorithm 15 K-PKE.Decrypt($\mathsf{dk}_{\mathsf{PKE}}, c$)

Uses the decryption key to decrypt a ciphertext.

Input: decryption key $\mathsf{dk}_{\mathsf{PKE}} \in \mathbb{B}^{384k}$.
Input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.
Output: message $m \in \mathbb{B}^{32}$.

1: $c_1 \leftarrow c[0 : 32d_u k]$
2: $c_2 \leftarrow c[32d_u k : 32(d_u k + d_v)]$
3: $\mathbf{u}' \leftarrow \mathsf{Decompress}_{d_u}(\mathsf{ByteDecode}_{d_u}(c_1))$ ▷ run $\mathsf{Decompress}_{d_u}$ and $\mathsf{ByteDecode}_{d_u}$ k times
4: $v' \leftarrow \mathsf{Decompress}_{d_v}(\mathsf{ByteDecode}_{d_v}(c_2))$
5: $\hat{\mathbf{s}} \leftarrow \mathsf{ByteDecode}_{12}(\mathsf{dk}_{\mathsf{PKE}})$ ▷ run $\mathsf{ByteDecode}_{12}$ k times
6: $w \leftarrow v' - \mathsf{NTT}^{-1}(\hat{\mathbf{s}}^{\top} \circ \mathsf{NTT}(\mathbf{u}'))$ ▷ run NTT k times; run NTT^{-1} once
7: $m \leftarrow \mathsf{ByteEncode}_1(\mathsf{Compress}_1(w))$ ▷ decode plaintext m from polynomial v
8: **return** m

$$
\begin{aligned}
w &= v - \mathbf{s}^T \mathbf{u} \\
&= \mathbf{t}^T \mathbf{y} + e_2 + \mu - \mathbf{s}^T (\mathbf{A}^T \mathbf{y} + \mathbf{e}_1) \\
&= (\mathbf{A}\mathbf{s} + \mathbf{e})^T \mathbf{y} + e_2 + \mu - \mathbf{s}^T \mathbf{A}^T \mathbf{y} + \mathbf{s}^T \mathbf{e}_1 \\
&= \mu + \mathbf{e}^T \mathbf{y} + e_2 + \mathbf{s}^T \mathbf{e}_1
\end{aligned}
$$

Internal Encaps & Decaps

Algorithm 17 ML-KEM.Encaps_internal(ek, m)

Uses the encapsulation key and randomness to generate a key and an associated ciphertext.

Input: encapsulation key $\text{ek} \in \mathbb{B}^{384k+32}$.
Input: randomness $m \in \mathbb{B}^{32}$.
Output: shared secret key $K \in \mathbb{B}^{32}$.
Output: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.

$(K, r) \leftarrow \text{G}(m \| \text{H}(\text{ek}))$ ▷ derive shared secret key K and randomness r
$c \leftarrow \text{K-PKE.Encrypt}(\text{ek}, m, r)$ ▷ encrypt m using K-PKE with randomness r
return (K, c)

Algorithm 18 ML-KEM.Decaps_internal(dk, c)

Uses the decapsulation key to produce a shared secret key from a ciphertext.

Input: decapsulation key $\text{dk} \in \mathbb{B}^{768k+96}$.
Input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.
Output: shared secret key $K \in \mathbb{B}^{32}$.

$\text{dk}_{\text{PKE}} \leftarrow \text{dk}[0 : 384k]$ ▷ extract (from KEM decaps key) the PKE decryption key
$\text{ek}_{\text{PKE}} \leftarrow \text{dk}[384k : 768k + 32]$ ▷ extract PKE encryption key
$h \leftarrow \text{dk}[768k + 32 : 768k + 64]$ ▷ extract hash of PKE encryption key
$z \leftarrow \text{dk}[768k + 64 : 768k + 96]$ ▷ extract implicit rejection value
$m' \leftarrow \text{K-PKE.Decrypt}(\text{dk}_{\text{PKE}}, c)$ ▷ decrypt ciphertext
$(K', r') \leftarrow \text{G}(m' \| h)$
$\bar{K} \leftarrow \text{J}(z \| c)$
$c' \leftarrow \text{K-PKE.Encrypt}(\text{ek}_{\text{PKE}}, m', r')$ ▷ re-encrypt using the derived randomness r'
if $c \neq c'$ **then**
    $K' \leftarrow \bar{K}$ ▷ if ciphertexts do not match, “implicitly reject”
end if
return K'

Parameter Sets of ML-KEM

	n	q	k	η_1	η_2	d_u	d_v	required RBG strength (bits)
ML-KEM-512	256	3329	2	3	2	10	4	128
ML-KEM-768	256	3329	3	2	2	10	4	192
ML-KEM-1024	256	3329	4	2	2	11	5	256

	encapsulation key	decapsulation key	ciphertext	shared secret key
ML-KEM-512	800	1632	768	32
ML-KEM-768	1184	2400	1088	32
ML-KEM-1024	1568	3168	1568	32

- Bit length of the numbers is 12 as q = 3329.
- Three security categories: 1, 3, 5.
- NIST recommends using ML-KEM-768 as the default parameter set.

Features of ML-KEM

- Sizes of public keys are significantly reduced by sending only the seed for generating the random matrix $\mathbf{A}$.
- Sizes of the ciphertexts are reduced by compression.
- SHAKE functions are heavily used to generate pseudorandom sequences.